V-01	**SQLi Login**	Consulta construida con f-string concatenando el input del usuario.	Reemplazar con consulta parametrizada usando ? como placeholder.

1. Descripción del problema
La funcionalidad de inicio de sesión presenta una vulnerabilidad de **SQL Injection**, ya que la consulta SQL se construye mediante concatenación directa de los datos ingresados por el usuario.
Esto permite que un atacante modifique la estructura de la consulta y altere su comportamiento original, logrando acceder al sistema sin credenciales válidas.

```python
cur.execute("SELECT COUNT(*) FROM users")
if cur.fetchone()[0] == 0:
    users = [
        ("admin",   "Admin123",   "admin"),
        ("analyst", "Analyst123", "user"),
        ("student", "Student123", "user"),
    ]
    cur.executemany(
        "INSERT INTO users (username, password, role) VALUES (?, ?, ?)",
        users
    )
```

```python
# ------------------------------------------------
# V-01: Consulta vulnerable en UNA SOLA LÍNEA para que el comentario
# SQL (--) funcione correctamente en SQLite y el payload surta efecto.
# Payload de ejemplo: usuario = admin' --  / password = (cualquier cosa)
query = f"SELECT id, username, role FROM users WHERE username = '{username}' AND password = '{password}'"

conn = get_connection()
try:
    user = conn.execute(query).fetchone()
except Exception as e:
    # El error de SQLite se muestra directamente - también
    # es información sensible que no debe exponerse.
    flash(f"Error en la base de datos: {e}", "error")
    conn.close()
    return render_template("login.html", last_query=query)
conn.close()
```

Se utilizó el siguiente payload en el campo de usuario:
- Usuario: admin' --
- Contraseña: cualquier valor

Este payload aprovecha el operador --, que en SQL indica un comentario, ignorando la validación de la contraseña.

2. ¿Por qué es peligrosa?
Esta vulnerabilidad es crítica porque permite:
- Acceso no autorizado al sistema
- Suplantación de identidad de usuarios (incluyendo administradores)
- Posible acceso a información sensible
- Escalamiento de privilegios

Además, un atacante podría utilizar técnicas más avanzadas para extraer información de la base de datos.

3. Corrección

```
conn = get_connection()
try:
    user = conn.execute(
        "SELECT * FROM users WHERE username = ?",
        (username,)
    ).fetchone()
except Exception:
    flash("Error en la base de datos.", "error")
    conn.close()
    return render_template("login.html")
conn.close()

from werkzeug.security import check_password_hash

if user and check_password_hash(user["password"], password):
    session["user_id"]  = user["id"]
    session["username"] = user["username"]
    session["role"]     = user["role"]
    log_event("LOGIN_OK", user["username"])
    flash("Inicio de sesión exitoso.", "success")
    return redirect(url_for("dashboard"))

log_event("LOGIN_FAIL", username)
flash("Credenciales incorrectas.", "error")
```

V-04	**SECRET_KEY fija**	La clave secreta está hardcodeada en el código fuente.	Cargarla desde una variable de entorno con os.environ.get().

1. Descripción del problema

La clave secreta está **escrita directamente en el código**.

```
# ------------------------------------------------------------
# V-04: SECRET_KEY hardcodeada en el código fuente.
# En una aplicación real debe cargarse desde una variable de
# entorno y nunca commitearse al repositorio.
# ------------------------------------------------------------
app.config["SECRET_KEY"] = "dev-secret-key-insegura-1234"
```

2. ¿Por qué es peligrosa?

- La SECRET_KEY se usa para:
    - Firmar sesiones
    - Cookies seguras

- Tokens

Si alguien la conoce, puede:

- Falsificar sesiones
- Hacerse pasar por otro usuario
- Escalar privilegios

3. Corrección

```
import sqlite3
import os
from pathlib import Path
```

```
# ------------------------------------------------------------
# V-04: SECRET_KEY hardcodeada en el código fuente.
# En una aplicación real debe cargarse desde una variable de
# entorno y nunca commitearse al repositorio.
# ------------------------------------------------------------
app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY", "dev_key")
#En terminal: set SECRET_KEY=mi_clave_super_segura
```

V-05	**debug=True**	El debugger de Flask está activo y permite ejecutar código en el servidor.	Cambiar a debug=False o leer el valor desde una variable de entorno.

1. Descripción del problema

En el código se encontró que la aplicación se ejecuta con el modo debug activado.
El modo debug en Flask está diseñado únicamente para desarrollo, ya que habilita un debugger interactivo en el navegador cuando ocurre un error.

```
# ------------------------------------------------------------
# V-05: debug=True activo.
# En modo debug, Flask activa un debugger interactivo en el
# navegador cuando ocurre un error. Cualquier visitante puede
# ejecutar código Python arbitrario en el servidor.
# Nunca debe usarse debug=True en producción.
# ------------------------------------------------------------
if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000, debug=True)
```

2. ¿Por qué es peligrosa?

Cuando debug=True está activo:

- Se muestra información interna del servidor
- Se exponen errores detallados

- Se habilita un debugger interactivo

Esto permite que un atacante pueda:

- Ejecutar código en el servidor
- Acceder a información sensible
- Manipular la aplicación

Es una vulnerabilidad crítica en entornos reales.

3. Corrección

```
app.run(
    host="0.0.0.0",
    port=5000,
    debug=os.environ.get("FLASK_DEBUG", "False") == "True"
)
```

El uso de variables de entorno permite habilitar el modo debug solo en entornos de desarrollo, evitando su uso en producción.

V-08	**Sin CSRF**	Los formularios no tienen token CSRF, permitiendo ataques de falsificación.	Instalar Flask-WTF, habilitar CSRFProtect y agregar {{ form.hidden_tag() }} o el token manual en cada formulario.

1. Descripción del problema

La aplicación no implementa protección contra ataques **CSRF (Cross-Site Request Forgery)** en sus formularios.

Esto significa que cualquier formulario (como login o búsqueda) puede ser enviado desde un sitio externo sin el consentimiento del usuario.

```
  <form method="post" autocomplete="off">
    <div class="form-group">
      <label for="username">Usuario</label>
      <input type="text" id="username" name="username" placeholder="Ingrese su usuario" required
    </div>
    <div class="form-group">
      <label for="password">Contraseña</label>
      <input type="password" id="password" name="password" placeholder="Ingrese su contraseña" re
    </div>
    <button type="submit" class="btn-primary">Iniciar sesión</button>
  </form>
</div>
```

2. ¿Por qué es peligrosa?

Un atacante puede:

- Hacer que un usuario autenticado ejecute acciones sin saberlo
- Enviar solicitudes maliciosas automáticamente
- Manipular el comportamiento de la aplicación

Ejemplo:
Un usuario logueado podría ser forzado a ejecutar acciones simplemente visitando una página maliciosa.

3. Corrección

```python
from flask_wtf.csrf import CSRFProtect
```

```python
app = Flask(__name__)
csrf = CSRFProtect(app)
```

```html
<div class="card">
<h2>Iniciar sesión</h2>
<form method="post" autocomplete="off">

  <!--Token CSRF -->
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

  <div class="form-group">
    <label for="username">Usuario</label>
    <input type="text" id="username" name="username" placeholder="Ingrese su usuario" required>
  </div>

  <div class="form-group">
    <label for="password">Contraseña</label>
    <input type="password" id="password" name="password" placeholder="Ingrese su contraseña" requ
  </div>

  <button type="submit" class="btn-primary">Iniciar sesión</button>
</form>
```